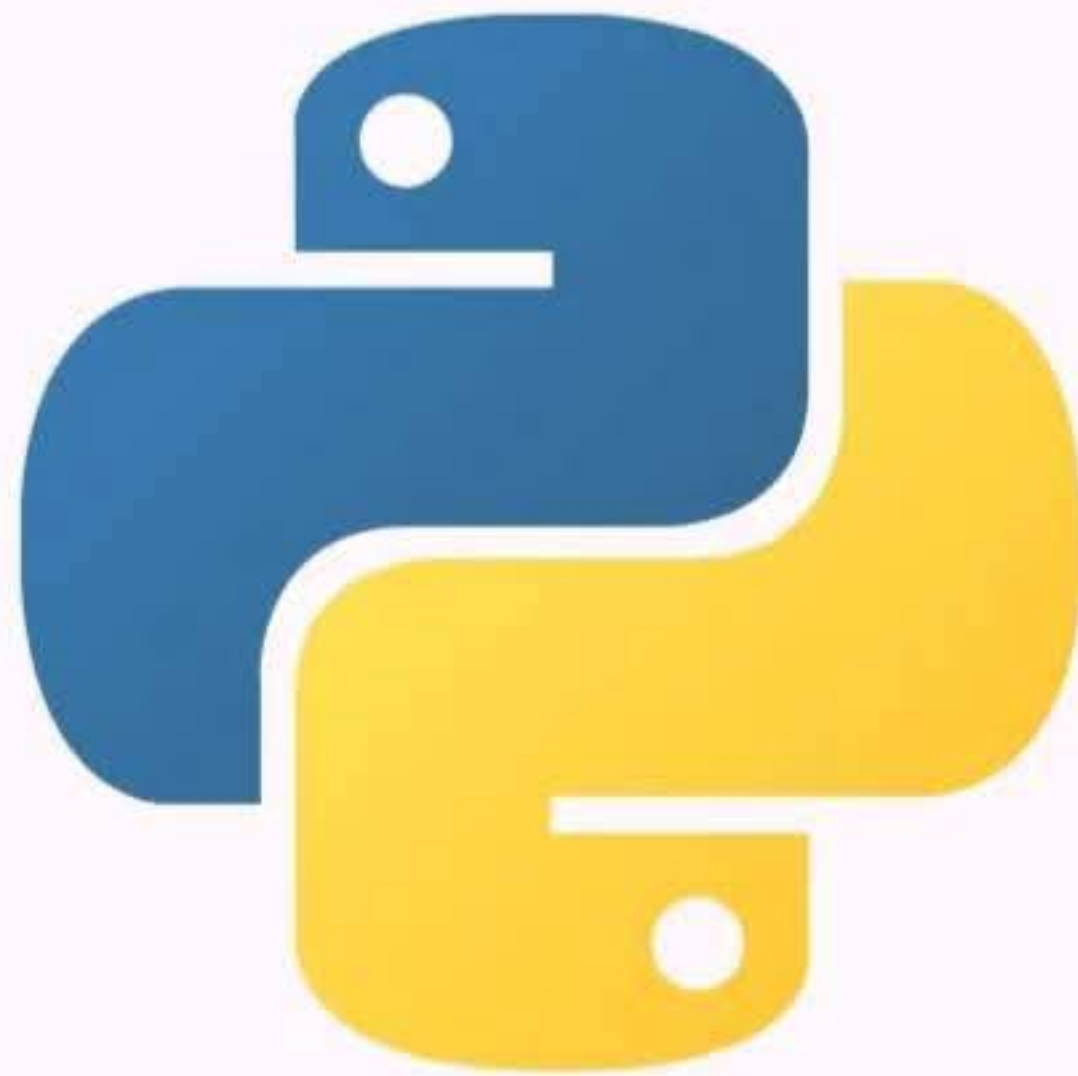


Python Fundamentals



[FOLLOW US](#)

Introduction to Python

What is Python?

Python is an interpreted, high-level, general-purpose programming language known for its simplicity and readability.

It's the most popular language in Data Science due to its rich ecosystem of libraries (Pandas, NumPy, scikit-learn).

Why Python for Data Science?

- Easy to learn & code
- Huge community support
- Extensive data science libraries
- Integration with other tools
(SQL, APIs, Dashboards)

Setting up Python

- **Install Anaconda** (recommended for beginners)
- **Alternative:** Install Python from python.org
- **Editors:** Jupyter Notebook (interactive), VS Code (lightweight)

Write Your First Program

```
print("Hello, Data Science World!")  
output:  
Hello, Data Science World!
```

Python Basics

Variables and Data Types

```
name = "Rohan"      # String  
age = 25             # Integer  
height = 5.6         # Float  
is_student = True    # Boolean
```

Operators

- Arithmetic: +, -, *, /, %, //, **
- Comparison: ==, !=, >, <, >=, <=
- Logical: and, or, not

Input & Output

```
name = input("Enter your name: ")  
print("Hello", name)
```

Comments & Docstrings

```
# This is a single-line comment  
""""  
  
This is a  
multi-line docstring.  
""""
```

Control Flow

Conditional Statements

```
age = 18
if age >= 18:
    print("Adult")
elif age > 13:
    print("Teenager")
else:
    print("Child")
```

Loops

For Loop

(General concept of repeating code.)

```
for i in range(5):
    print(i)
```

While Loop

(A specific way to repeat code while a condition stays True.)

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Loop Control Statements

```
for i in range(10):
    if i == 5:
        continue # skips 5
    if i == 8:
        break # stops loop at 8
    print(i)
```

Data Structures in Python

Lists

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0])           # Access  
fruits.append("orange")    # Add
```

Ordered, mutable
(changeable), allows
duplicates

Tuples

```
coordinates = (10, 20)  
print(coordinates[0])
```

Ordered, immutable
(cannot change),
allows duplicates.

Dictionaries

```
student = {"name": "John", "age": 22}  
print(student["name"])
```

unordered, no
duplicates, mutable (can
add/remove elements).

Sets

```
unique_numbers = {1, 2, 3, 2, 1}  
print(unique_numbers) # Output: {1, 2, 3}
```

Key-Value pairs,
unordered (Python 3.7+
maintains insertion order)
mutable.

List Comprehensions

```
squares = [x**2 for x in range(5)]  
print(squares)
```

One-line syntax to
create new lists by
looping & filtering.

Functions and Modules

Defining Functions

```
def greet(name):  
    return f"Hello {name}"  
print(greet("Alice"))
```

A function is a block of reusable code that performs a specific task.

Arguments

```
def add(a, b=5):  
    return a + b  
print(add(10)) # Uses default b=5
```

Lambda Functions

```
square = lambda x: x**2  
print(square(4))
```

is a small, one-line, anonymous function.

Importing Modules

```
import math  
print(math.sqrt(16))
```

Modules are Python files (.py) containing functions, classes, variables, etc.

File Handling

Reading Files

```
with open('data.txt', 'r') as file:  
    content = file.read()  
    print(content)
```

Writing to Files

```
with open('output.txt', 'w') as file:  
    file.write("This is a new file.")
```

Common Mistakes to Avoid

1. Forgetting to close the file → Use with open(...) to handle it.
2. Writing in 'r' mode → Use 'w' or 'a' for writing.
3. FileNotFoundError → Ensure the file path is correct for reading.

Exception Handling

Try-Except Block

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero!")
```

Finally Block

```
try:  
    file = open('file.txt', 'r')  
except FileNotFoundError:  
    print("File not found")  
finally:  
    print("Execution complete")
```

Raising Exceptions

```
def check_age(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative")
```

You've now covered:

- Python setup & syntax
- Data types & operations
- Control flow (conditions, loops)
- Core data structures
- Functions, file handling
- Exception management

This is your foundation block to move towards Data Science-specific libraries (Pandas, NumPy) and hands-on data projects.

Next Up →

**Part 2: Data Science
Essentials with Python**